

Rule 37R on a Torus: Searching for the Longest Repeating Spacetimes

Exploration with Grok

The Program

We study **Wolfram’s reversible elementary cellular automaton Rule 37R** on a one-dimensional **torus** (periodic boundary conditions) with the standard initial condition $prev = 0$ (all zeros at time $t = -1$).

The program performs a full exhaustive search over all 2^N possible initial seeds and determines, for each N , the **longest possible repeat period** (cycle length) on the torus. It then reports the number of **distinct patterns up to cyclic rotation** that achieve this maximum period.

The core update rule is

$$next(i) = Rule37(left, center, right) \oplus prev(i)$$

with periodic boundaries.

Results

The following table shows the longest observed period and the number of distinct (rotationally inequivalent) initial seeds that achieve it:

N	Longest period	Total seeds on max cycles	Distinct up to rotation
6	32	12	2
9	585	24	4
10	782	120	12
12	4100	96	8
16	77648	256	16

Global Constraints and “Most Interesting” Spacetimes

Every finite 37R spacetime on a torus **must eventually repeat**. The total number of possible states is 2^{2N} (a pair of N -bit rows), therefore the theoretical upper bound on any cycle length is

$$2^{2N}.$$

In practice the longest cycles are far shorter. The search for the **longest possible torus** acts as a strong global constraint: it selects only those very rare initial conditions that produce the “most interesting” spacetimes — the ones that visit the largest number of distinct configurations before repeating.

This illustrates a beautiful general principle in cellular automata: **global constraints (here, maximising the repeat period) can dramatically reduce the space of possible behaviours**, leaving only a tiny handful of highly structured, long-lived spacetimes.

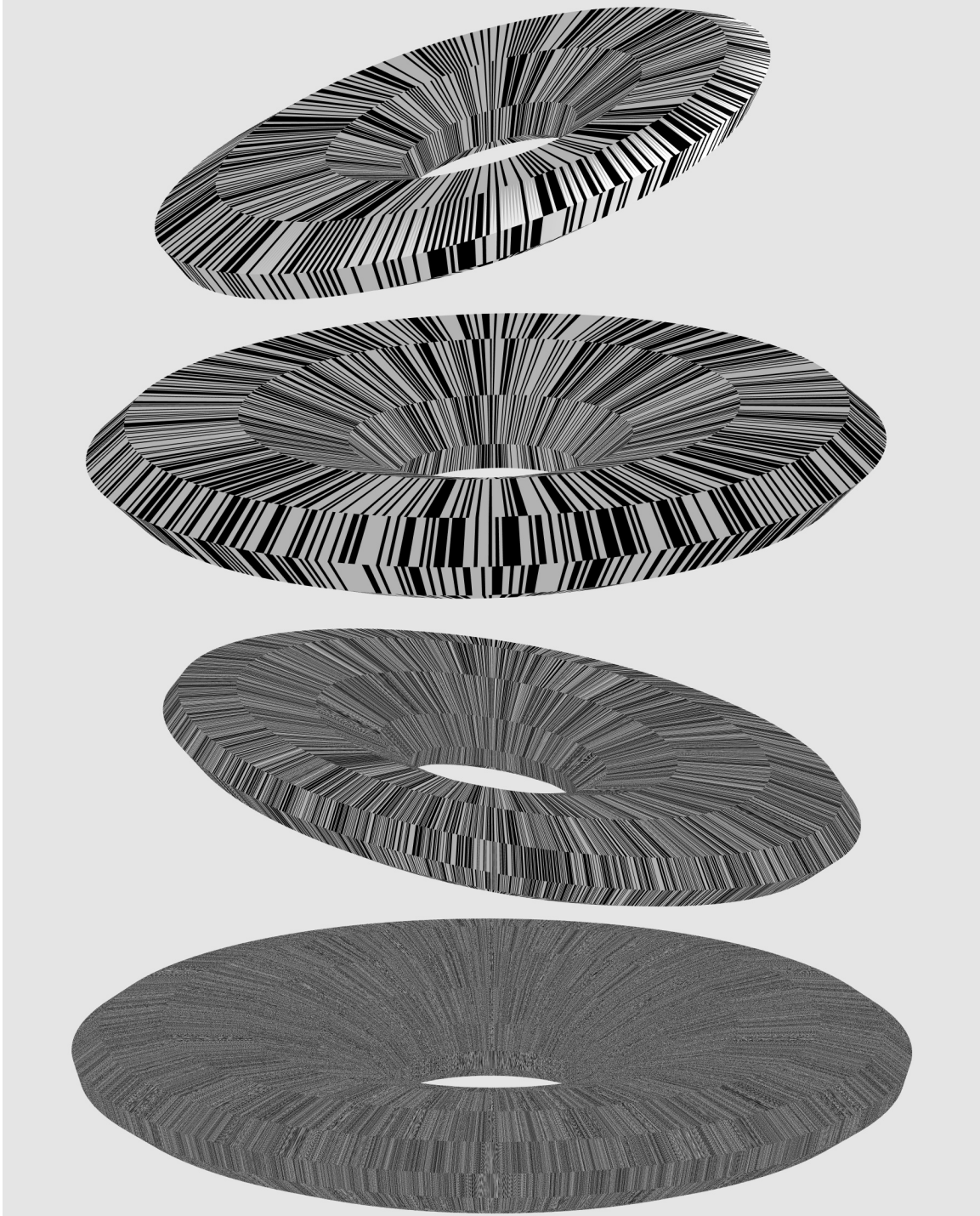


Figure 1: Visualisation of the longest cycles for $N = 9, 10, 12, 16$ (thick-torus view). All tori are scaled/normalized to fit comfortably on the screen.

The program and all results were obtained with an optimised exhaustive Python searcher combined with an interactive Plotly 3D torus visualisation.